

Smart Parking System - Database Normalization

Group 2

Course: Database systems(SWEG2108)

Instructor: Yaynshet Medhin

1. Unnormalized Form (UNF) - Initial Data

Before normalization, all data is gonna be stored in a single table with repeating groups:

Field	Contains
username, password, role, isVIP	User details
plate_number, vehicle_type	Vehicle details
slot_id, slot_type, price_per_hour	Parking slot details
booking_id, start_time, end_time, total_fee, status	Booking details
payment_id, amount, payment_status	Payment details

Problems:

- Repeating groups (a user can have multiple vehicles and bookings)
- Data redundancy
- Update anomalies
- Insertion anomalies
- Deletion anomalies

2. First Normal Form (1NF)

Rule: Eliminate repeating groups. Ensure each column contains atomic (indivisible) values. Each table must have a primary key.

Transformation Applied:

Separated repeating groups into distinct entities

Identified primary keys for each entity

Ensured all attributes are atomic values

Resulting Entities (1NF):

Entity	Primary Key	Attributes
User	user_id	username, password_hash, role, is_vip
Vehicle	plate_number	user_id (FK), vehicle_type

Entity	Primary Key	Attributes
ParkingSlot	slot_id	slot_type, price_per_hour, is_occupied
Booking	booking_id	user_id (FK), plate_number (FK), slot_id (FK), start_time, end_time, total_fee, status
Payment	payment_id	booking_id (FK), amount, payment_status

1NF Status: Complete No repeating groups, all values atomic, primary keys defined.

3. Second Normal Form (2NF)

Rule: Meet all 1NF requirements & remove partial dependencies. A partial dependency exists when a non-key attribute depends on only part of a composite primary key, rather than the whole other keys.

Analysis:

Since all our entities from 1NF have **single-column primary keys** (not other composite keys), partial dependencies cannot exist in our schema:

Entity	Primary Key	Composite?	Partial Dependencies?
User	user_id	No (single)	None possible
Vehicle	plate_number	No (single)	None possible
ParkingSlot	slot_id	No (single)	None possible
Booking	booking_id	No (single)	None possible
Payment	payment_id	No (single)	None possible

Verdict:

No changes needed. The schema naturally satisfies 2NF because no entity uses a composite primary key. Every non-key attribute depends on the entire primary key of its respective table.

Validation Example — Booking Entity:

- `start_time` → depends on full `booking_id`
- `total_fee` → depends on full `booking_id`
- `status` → depends on full `booking_id`

No attribute depends on only *part* of a `booking_id`, since `booking_id` is a single column.

2NF Status: done No partial dependencies exist.

4. Third Normal Form (3NF)

Rule: Meet all 2NF requirements AND remove transitive dependencies. A transitive dependency exists when a non-key attribute depends on another non-key attribute, rather than directly on the primary key.

Analysis:

We examined each entity for transitive dependencies:

User Entity:

Attribute	Depends On	Issue?
username	user_id	Direct
password_hash	user_id	Direct
role	user_id	Direct
is_vip	user_id	Direct

Verdict: no transitive dependencies.

Vehicle Entity:

Attribute	Depends On	Issue?
user_id	plate_number	Direct (FK)
vehicle_type	plate_number	Direct

Verdict: no transitive dependencies.

ParkingSlot Entity:

Attribute	Depends On	Issue?
slot_type	slot_id	Direct
price_per_hour	slot_id	Direct
is_occupied	slot_id	Direct

Verdict: no transitive dependencies.

Booking Entity:

Attribute	Depends On	Issue?
user_id	booking_id	Direct (FK)
plate_number	booking_id	Direct (FK)
slot_id	booking_id	Direct (FK)
start_time	booking_id	Direct
end_time	booking_id	Direct
total_fee	booking_id	Direct

Attribute	Depends On	Issue?
status	booking_id	Direct

Verdict: no transitive dependencies.

Payment Entity:

Attribute	Depends On	Issue?
booking_id	payment_id	Direct (FK)
amount	payment_id	Direct
payment_status	payment_id	Direct

Verdict: no transitive dependencies.

Transitive Dependency Check — Deeper Analysis:

We also verified that:

- `price_per_hour` stays in **ParkingSlot**, NOT in Booking — avoids duplicating price data
- `amount` stays in **Payment**, calculated from Booking duration + ParkingSlot rate — not stored redundantly
- User's `is_vip` doesn't get copied to Booking — referenced via FK to User table

Verdict:

No changes needed. The schema naturally satisfies 3NF. Every non-key attribute depends directly on the primary key of its table, not on another non-key attribute.

3NF done to make sure there is no transitive dependencies exist.

5. Final Normalized Schema (3NF)

The Smart Parking System database is fully normalized to Third Normal Form (3NF):

Users Table

Column	Type	Key
user_id	Integer	Primary Key
username	String	Unique
password_hash	String	
role	String (Admin/Client)	
is_vip	Boolean	

Vehicles Table

Column	Type	Key
plate_number	String	Primary Key
user_id	Integer	Foreign Key (Users)
vehicle_type	String	

Parking_Slots Table

Column	Type	Key
slot_id	String	Primary Key
slot_type	String (Normal/VIP)	
price_per_hour	Float	
is_occupied	Boolean	

Bookings Table

Column	Type	Key
booking_id	Integer	Primary Key
user_id	Integer	Foreign Key (Users)
plate_number	String	Foreign Key (Vehicles)
slot_id	String	Foreign Key (Parking_Slots)
start_time	DateTime	
end_time	DateTime	
total_fee	Float	
status	String (Active/Completed/Cancelled)	

Payments Table

Column	Type	Key
payment_id	Integer	Primary Key
booking_id	Integer	Foreign Key (Bookings)
amount	Float	
payment_status	String (Pending/Paid)	
paid_at	DateTime	

Entity-Relationship Summary:

Relationship	Type	Description
User → Vehicle	One-to-Many	A user can own multiple vehicles
User → Booking	One-to-Many	A user can make multiple bookings
ParkingSlot → Booking	One-to-Many	A slot can be booked multiple times
Booking → Payment	One-to-One	Each booking has exactly one payment

Foreign Key References:

Table	Foreign Key	References
Vehicles	user_id	Users(user_id)
Bookings	user_id	Users(user_id)
Bookings	plate_number	Vehicles(plate_number)
Bookings	slot_id	Parking_Slots(slot_id)
Payments	booking_id	Bookings(booking_id)

6. Conclusion

The Smart Parking System database has been successfully normalized through all three normal forms:

Normal Form	Status	Changes Made
1NF	Complete	Separated single table into 5 entities with atomic values and primary keys
2NF	Complete	Verified that there is no partial dependencies exist (all PKs are single-column)
3NF	Complete	Verified that there is no transitive dependencies; all attributes depend directly on PKs

Benefits Achieved:

- **Data Integrity:** Each piece of information is stored exactly once which eliminats inconsistencies
- **No Redundancy:** User details, slot prices, and vehicle info are never duplicated
- **Update Efficiency:** Changing a user's VIP status or a slot's price requires updating only one row
- **Deletion Safety:** Deleting a booking does not erase user or vehicle records
- **Scalability:** New entities can be added without needing to restruct existing tables

The normalized schema provides a solid relational database foundation for the Smart Parking System, ensuring efficient querying, data consistency, and ease of maintenance.